

INTRODUCTION TO PROGRAMMING (CSE141)

Lab Examination – Set A

Max Marks: 40 | Time Allowed: 2 Hours

Submission folder: Name_ERP (e.g., Ali_20261234). Save all four specified .cpp files in this folder.

Exercise 1. The Rising Shuttle Streak

10 marks

The IBA City Campus shuttle manager records the number of passengers travelling on the shuttle for **10 consecutive days**. Write a C++ program that stores these passenger counts in a **one-dimensional array of size 10** and finds the longest consecutive period during which the number of passengers increases every day.

Your program should display:

1. The length of the longest increasing period (number of days).
2. The starting and ending day numbers of that period.

Requirements:

- Use a fixed-size integer array of 10 elements.
- Read the passenger counts for all 10 days.
- A period is increasing only when each day's passenger count is strictly greater than the previous day's count.
- A period of one day is also valid.
- If multiple periods have the same maximum length, you may report any one of them.
- Do not sort or rearrange the passenger counts.
- Days are numbered starting from 1.

Submission file: shuttle.cpp

Sample Input — Keyboard input

```
10 13 12 14 16 18 17 19 21 20
```

Sample Output — Console output

```
Longest streak: 4
Days: 3 to 6
```

Marking rubric

Marking Criteria	Marks
Reading and storing 10 values in an array	1
Identifying consecutive increases	4
Finding the longest increasing period	4
Displaying correct day numbers and output	2

Exercise 2. The SMCS Display Wall — Case Study

10 marks

The SMCS laboratory has a digital display consisting of **3 rows and 3 columns** of numbered tiles.

Write a C++ program that reads 9 integers into a two-dimensional array of size 3×3 and **rotates the display 90 degrees clockwise**.

After rotation, your program should display:

1. The rotated 3×3 matrix.
2. The sum of the elements on its main diagonal (top-left to bottom-right).

Sample Before (2×2):

1 2

3 4

AFTER:

3 1

4 2

Input: n (1 to 10), followed by n rows of n integers. Print the resulting $n \times n$ layout, followed by (i) the sum of its secondary diagonal. (ii) Also mention if secondary diagonal is equal to main diagonal, greater or less than it.

Note: You must determine the transformation yourself; no sequence of steps is supplied.

Submission file: `display.cpp`

Sample Input — Keyboard input

```
1 2 3
4 5 6
7 8 9
```

Sample Output — Console output

```
After:
7 4 1
8 5 2
9 6 3
Secondary diagonal sum: 15
Secondary and Main Diagonal are Equal.
```

Marking rubric

Assessment criterion	Marks
Secondary diagonal total	4
Main diagonal total	3
Comparison of main and secondary diagonal	2
Correct output formatting	1
TOTAL	10

Exercise 3. Karachi Entry Code Audit

10 marks

A visitor pass has exactly eight characters. Its first three characters must be UPPERCASE English letters, its fourth character must be a hyphen (-), and its final four characters must be digits. The LAST digit must equal the sum of the PREVIOUS three digits modulo 10. Any failure makes the whole pass invalid.

Input: n (1 to 100), then n pass tokens, each at most 30 characters long and without spaces. Read each pass into a fixed-size character array. Check the characters individually, without string-library functions. Print one decision per pass, and then the total accepted count. Case matters.

Submission file: codes.cpp

Sample Input — Keyboard input

```
4
KHI-4824
LHR-2350
khi-4824
KHI-4821
```

Sample Output — Console output

```
KHI-4824: VALID
LHR-2350: VALID
khi-4824: INVALID
KHI-4821: INVALID
Accepted: 2
```

Marking rubric

Assessment criterion	Marks
Exact eight-character length and format	3
Digit and uppercase validation	2
Modulo-10 check	3
Correct decisions and total	2
TOTAL	10

Exercise 4. Cafeteria Revenue Ranking — Files

10 marks

The IBA cafeteria exports a file named `sales.txt`. Its first number is n (2 to 100), followed by n records containing a unique positive stall ID and its nonnegative integer revenue (each at most 1,000,000). Read the FILE, then write ALL records into `ranking.txt` ordered by HIGHER revenue first. If revenues tie, the LOWER stall ID comes first. Also print how many records were written and the ARITHMETIC MEAN of the first TWO ranked revenues on the console. Use explicit floating-point casting so any fractional part is not lost. Use fixed-size arrays and any sorting algorithm covered in class. Assume `sales.txt` exists in the program's working folder.

Each line of `ranking.txt` must contain an ID, one space, and its revenue; no heading in the output file.

Submission file: `revenue.cpp`

Sample Input — File `sales.txt` (input)

```
6
104 125
101 80
107 125
102 210
105 80
109 175
```

Sample Output — File `ranking.txt` (output)

```
102 210
109 175
104 125
107 125
101 80
105 80
```

Sample Output — Console

```
Records written: 6
Top-two mean: 192.5
```

Marking rubric

Assessment criterion	Marks
Read and validate the prescribed file layout	
Correct two-key selection/ranking	
Write exact file content	
Console summary	
TOTAL	